

INFOSYS SPRINGBOARD INTERNSHIP 7.0

PROJECT DOCUMENTATION REPORT

**ORGANIZATIONAL KNOWLEDGE GAP
INTELLIGENCE PLATFORM**

DOMAIN: JAVA FULL STACK TECHNOLOGY

Submitted by:

Likhith Sai Bandaru

Email Id: bandarulikhithsai@gmail.com

TEAM - 01

BATCH - 01

ABSTRACT

Organizations bring together employees with different combinations of technical skills, professional competencies and practical experience, distributed across roles, teams and departments. As an organization grows, maintaining an accurate and current understanding of workforce capabilities becomes increasingly difficult using manual assessments and spreadsheets. This makes it hard for managers and HR teams to identify critical knowledge gaps and to direct learning interventions to where they are actually needed.

The Organizational Knowledge Gap Intelligence Platform is a full-stack web application developed by Team 1 as part of the Infosys Springboard Internship 7.0 to address this problem. The system combines a React.js frontend with a Spring Boot backend and a PostgreSQL database to provide a single environment where employees, managers, HR professionals and administrators can maintain skill data, benchmark it against role requirements, and act on the resulting insights.

The platform maintains employee profiles and skill inventories and compares them against role-specific and department-specific competency requirements defined by the organization. From this comparison, the Knowledge Gap Analysis module identifies deficiencies at the individual, team and department level, assigns a severity to each gap, and presents the results through dashboards and heatmap-style visualizations so that stakeholders can quickly see where attention is required.

Identifying a gap is only the first step, so the platform connects each identified gap to a corresponding learning intervention. The Training Recommendation module applies rule-based matching between an employee's skill gaps and a catalog of training programs to suggest relevant courses and learning paths. The platform additionally supports mentorship and knowledge-sharing between employees, tracks learning progress and training enrollment, and provides self, peer and manager assessment workflows that feed back into the gap analysis. Notifications are generated and stored within the application to keep employees and managers informed of relevant events, and role-based analytics dashboards and exportable reports give managers, HR and administrators an organization-wide view of skill health.

The result is an integrated cycle in which employee skills are captured, gaps are identified and prioritized, learning interventions are recommended, progress is tracked, and organizational insight is generated through analytics and reporting, rather than treating these as separate, disconnected activities. This report documents the objectives, architecture, database design, module-wise implementation, testing approach, challenges and outcomes of the platform as developed by Team 1.

TABLE OF CONTENTS

- Abstract.....2**
- List of Figures.....4**
- List of Tables5**
- 1. Project Overview and Objectives6**
 - 1.1 Introduction6
 - 1.2 Problem Statement.....6
 - 1.3 Project Motivation6
 - 1.4 Proposed Solution.....6
 - 1.5 Core Objectives7
 - 1.6 Scope of the Platform7
 - 1.7 Key Outcomes7
 - 1.8 Project Development Milestones.....7
- 2. System Requirements and Technology Stack8**
 - 2.1 System Overview.....8
 - 2.2 Functional Requirements.....8
 - 2.3 Non-Functional Requirements.....9
 - 2.4 Frontend Technology Stack.....9
 - 2.5 Backend Technology Stack10
 - 2.6 Technologies Considered but Not Implemented10
 - 2.7 Development and Testing Tools.....10
- 3. System Architecture and Technical Design11**
 - 3.1 Architecture Overview11
 - 3.2 Client Layer11
 - 3.3 Security Layer11
 - 3.4 Backend Application Layer.....12
 - 3.5 Data Storage Layer12
 - 3.6 Control Flow.....12
 - 3.7 Data Flow: Gap Analysis to Recommendation13
 - 3.8 Communication Model.....13
- 4. Database and Data Design.....14**
 - 4.1 Database Overview.....14
 - 4.2 Entity Relationship Diagram14
 - 4.3 Core Database Entities.....15
 - 4.4 User and Profile Data15
 - 4.5 Skills and Competency Data.....15
 - 4.6 Knowledge Gap Data16
 - 4.7 Training and Progress Data16
 - 4.8 Assessment, Mentorship and Notification Data16
 - 4.9 Database Design Considerations16

5. Module-wise Implementation 17
 5.1 – 5.12 Modules 1 through 12..... 17
 5.13 Module Summary 22
6. Testing, Validation and Code Quality..... 23
7. Challenges and Future Enhancements 25
8. Project Outcomes, Team Contributions and Conclusion..... 27
9. References and Appendices..... 29

LIST OF FIGURES

Figure 3.1: System Architecture of the Organizational Knowledge Gap Intelligence Platform

Figure 3.2: Control Flow and Component Interaction Diagram

Figure 3.3: Data Flow Diagram for Knowledge Gap Analysis and Training Recommendation

Figure 4.1: Entity Relationship Diagram of the Platform Database

Figure 5.1: Application login screen with role-based demo accounts

Figure 5.2: Employee skill inventory with self-assessed proficiency levels

Figure 5.3: Competency framework showing required proficiency per role

Figure 5.4: Current vs. required proficiency chart for an individual employee

Figure 5.5: Organisational gap heatmap by department

Figure 5.6: Gap detail table with automated severity scoring

Figure 5.7: Recommended training programs ranked by gap-closure match

Figure 5.8: System-generated learning path sequenced by gap severity

Figure 5.9: Matched mentors ranked by skill-complementarity score

Figure 5.10: Knowledge-sharing sessions available for registration

Figure 5.11: Knowledge articles shared across the community of practice

Figure 5.12: Enrollment progress tracking with completion controls

Figure 5.13: Skill-tagged assessment quiz catalog

Figure 5.14: Core Java Proficiency quiz in progress

Figure 5.15: Notifications inbox showing gap alerts and training deadlines

Figure 5.16: Employee dashboard: knowledge gap summary with prioritised gaps

Figure 5.17: Employee dashboard: recommended training and learning progress

Figure 5.18: Analytics view: top organisational skill gaps and department readiness

Figure 5.19: Analytics view: individual progress snapshots across the organisation

Figure 5.20: Analytics view: team gap heatmap by employee and skill

LIST OF TABLES

- Table 2.1:** Functional Requirements Summary
- Table 2.2:** Frontend Technology Stack
- Table 2.3:** Backend Technology Stack
- Table 4.1:** Core Database Entities
- Table 5.1:** Module Summary
- Table 6.1:** Testing Approach by Layer
- Table 8.1:** Team Responsibilities

1. PROJECT OVERVIEW AND OBJECTIVES

1.1 Introduction

Employee knowledge and expertise are core resources within any organization. Employees carry different combinations of technical skills, professional competencies and practical experience, and these capabilities are unevenly distributed across teams and departments. Keeping this distribution aligned with current responsibilities and project requirements becomes progressively harder as an organization grows, as skills change over time and different roles come to require different proficiency levels.

Without a structured way of continuously assessing and analyzing workforce capabilities, gaps in a team's or department's skill coverage tend to remain unnoticed until they affect delivery or performance. The Organizational Knowledge Gap Intelligence Platform was built to address this by providing a centralized system for maintaining employee skill information and analyzing organizational competency gaps against defined role and department requirements.

1.2 Problem Statement

Conventional approaches to workforce skill tracking rely on manual assessments, spreadsheets and periodic reviews. These methods do not provide a centralized, continuously updated view of organizational capability, make it difficult to compare an employee's actual skill level against the level expected for their role, and offer no direct mechanism to connect an identified gap to an appropriate learning intervention. As a result, skill gaps are often identified late, addressed inconsistently, or not tracked to resolution.

1.3 Project Motivation

The project was motivated by the need for a single platform that unifies skill data capture, gap identification and learning intervention rather than treating them as separate, manually coordinated activities. Bringing these functions together allows an organization to move from a static record of who has which skill to an ongoing, analyzable view of where competency gaps exist and whether learning interventions are closing them.

1.4 Proposed Solution

The platform is a full-stack web application with a React.js frontend and a Spring Boot backend, backed by a PostgreSQL database. Employees maintain profiles and skill inventories; organizational administrators define roles, departments and the competencies each role requires. The system compares the two to compute knowledge gaps, which are then used to drive rule-based training recommendations, learning-progress tracking, mentorship connections, assessments, notifications, analytics dashboards and exportable reports.

1.5 Core Objectives

- Provide a centralized system for maintaining employee profiles and skill inventories.

- Define role- and department-level competency requirements against which employee skills can be benchmarked.
- Automatically identify and prioritize knowledge gaps at individual, team and department level.
- Recommend relevant training programs based on identified skill gaps.
- Support mentorship and knowledge-sharing between employees with complementary skills.
- Track learning progress, training enrollment and assessment outcomes over time.
- Provide role-based analytics dashboards and exportable reports for managers, HR and administrators.
- Enforce secure, role-based access to platform functionality.

1.6 Scope of the Platform

The platform scope covers four user roles — Employee, Manager, HR/Administrator and, where applicable, Department Head — and the full cycle from skill data entry through gap analysis, training recommendation, learning-progress tracking and reporting. The platform was developed and demonstrated as a web application; native mobile applications, external LMS integrations and organization-wide single sign-on are outside the scope of the current implementation and are discussed as future enhancements in Chapter 7.

1.7 Key Outcomes

The team delivered a working full-stack application covering all twelve planned modules, described in detail in Chapter 5, with a functioning authentication and role-based access layer, a relational database schema supporting the full skill-to-training lifecycle, and a rule-based recommendation engine connecting identified gaps to a training catalog. The platform was integrated end-to-end and demonstrated using realistic sample organizational data.

1.8 Project Development Milestones

The project progressed through five broad milestones: establishing requirements and the overall architecture; building the authentication and core backend foundation; implementing skill inventory, competency benchmarking and knowledge gap analysis; building training recommendation, mentorship, progress tracking and assessment functionality; and finally integrating notifications, analytics, reporting and completing end-to-end testing. Team contributions against these milestones are summarized in Chapter 8.

2. SYSTEM REQUIREMENTS AND TECHNOLOGY STACK

2.1 System Overview

The platform is a three-tier web application. The presentation tier is a React.js single-page application; the application tier is a Spring Boot backend exposing REST APIs and enforcing authentication, authorization and business logic; and the data tier is a PostgreSQL relational database accessed through Spring Data JPA and Hibernate. Communication between the frontend and backend uses synchronous HTTPS/REST requests; no message broker or asynchronous processing layer was used.

2.2 Functional Requirements

Table 2.1 summarizes the functional requirements the platform was built to satisfy, grouped by the module that implements them. Detailed module-level functionality is covered in Chapter 5.

Requirement Area	Description
Authentication & Access Control	Secure login and role-based access for Employee, Manager, HR/Admin roles.
Profile & Skill Inventory	Employees maintain profile details and a structured skill inventory with proficiency levels.
Competency Benchmarking	Administrators define required skill levels per role and department.
Knowledge Gap Analysis	System computes and prioritizes gaps between actual and required skill levels.
Training Recommendation	Rule-based matching of identified gaps to relevant training programs.
Mentorship & Knowledge Sharing	Employees with complementary skills can be connected for mentorship.
Learning Progress Tracking	Enrollment and progress against training programs is tracked over time.
Assessment & Survey	Self, peer and manager assessments feed back into skill and gap data.
Notification	In-application notifications for relevant events.
Analytics & Reporting	Role-based dashboards and exportable reports.

Table 2.1: Functional Requirements Summary

2.3 Non-Functional Requirements

2.3.1 Security

Access to the platform is protected by JWT-based authentication issued at login, with authorization enforced per endpoint according to the authenticated user's role.

2.3.2 Usability

The frontend was designed with a consistent component library and a dedicated navigation shell per role so that each user type is presented only with functionality relevant to them.

2.3.3 Maintainability

The backend follows a layered architecture (controller, service, repository) and the frontend uses a reusable component library, which keeps module-specific logic isolated and easier to extend.

2.3.4 Data Consistency

Relational integrity between employees, skills, competencies, gaps and training data is enforced through PostgreSQL foreign-key constraints, ensuring that gap and recommendation data always reference valid skill and profile records.

2.4 Frontend Technology Stack

Technology	Purpose
React.js (Vite)	Component-based single-page application framework and build tooling.
React Router DOM	Client-side routing between role-specific pages.
Tailwind CSS	Utility-first styling and design system tokens.
Axios	REST API communication with the backend, including a token-expiry interceptor.
lucide-react	Icon set used across the component library.

Table 2.2: Frontend Technology Stack

2.5 Backend Technology Stack

Technology	Purpose
Java	Primary backend programming language.
Spring Boot	Application framework used to build REST APIs and manage application configuration.
Spring Security	Authentication and role-based authorization, implemented with JWT.
Spring Data JPA / Hibernate	Object-relational mapping and repository-based data access.
Maven	Build and dependency management.
PostgreSQL	Relational database used for all persistent application data.

Table 2.3: Backend Technology Stack

2.6 Architecture Decisions and Deferred Scope

During architecture planning, the team evaluated several additional components used in comparable enterprise platforms: Elasticsearch for search, Redis for caching, a message broker (Kafka/RabbitMQ) for asynchronous processing, an API gateway (e.g. Spring Cloud Gateway) in front of the backend, Docker-based containerization with Nginx, Google OAuth2 as an additional login method, and third-party notification channels such as Twilio SMS and Firebase Cloud Messaging. Given the team size, the eight-week timeline, and the scale at which the

platform was demonstrated, these were deliberately scoped out rather than partially implemented: a single Spring Boot application serving REST endpoints directly keeps the system easier to build, test, and reason about at this scale, and avoids the operational overhead of running and coordinating additional infrastructure the team could not fully exercise or validate within the project timeline. In their place, search and filtering are handled through direct Spring Data JPA queries, all processing is synchronous within the Spring Boot application, authentication is JWT-only, notifications are generated and stored in-application, and the application was run and demonstrated locally rather than deployed to a containerized or cloud environment. Each of these is revisited as a concrete, scoped future enhancement in Chapter 7, where the specific architectural change required to introduce it is described.

2.7 Development and Testing Tools

Development used standard IDE tooling for Java/Spring Boot and JavaScript/React, Maven for backend builds, npm/Vite for the frontend build pipeline, and Postman for manual REST API validation during development. The testing approach is described in Chapter 6.

3. SYSTEM ARCHITECTURE AND TECHNICAL DESIGN

3.1 Architecture Overview

The platform follows a layered three-tier architecture: a React.js client layer, a Spring Boot application layer that includes a security layer for authentication and authorization, and a PostgreSQL data storage layer. There is no separate API gateway, message broker or caching layer in the current implementation — the Spring Boot application itself exposes REST endpoints directly to the frontend and handles all requests synchronously. This single-application design was a deliberate scope decision rather than an oversight: it was the architecture the team could fully build, test, and demonstrate within the project timeline, and Section 2.6 details the specific components (API gateway, message broker, caching layer) that a larger-scale deployment would introduce.

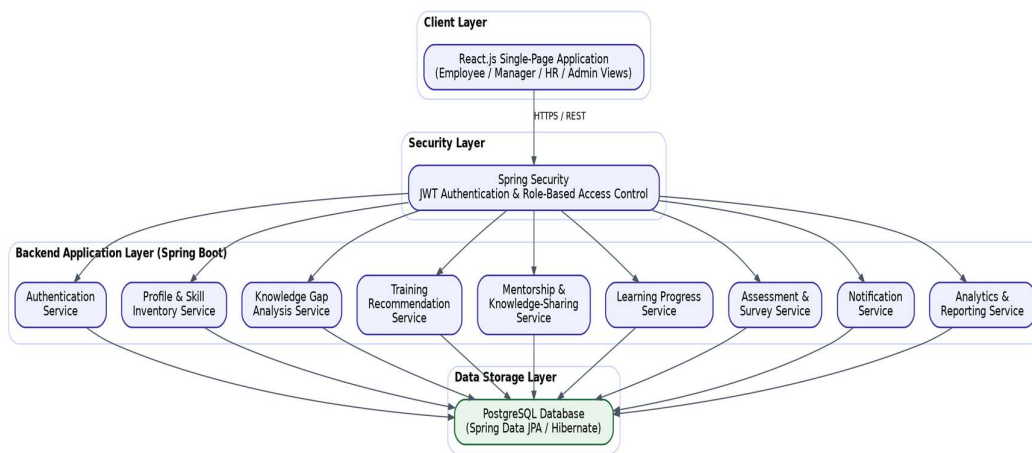


Figure 3.1: System Architecture of the Organizational Knowledge Gap Intelligence Platform

3.2 Client Layer

The client layer is a React.js single-page application built with Vite. It is organized around role-specific views for Employees, Managers and HR/Administrators, backed by a shared component library (buttons, modals, badges, stat cards, skeleton loaders) and a dedicated navigation shell. Routing between pages is handled by React Router DOM, and all backend communication goes through a centralized Axios API client that attaches the JWT access token to outgoing requests and intercepts 401 responses to handle token expiry.

3.3 Security Layer

3.3.1 JWT Authentication

Users authenticate with an email/password login handled by Spring Security, which issues a signed JWT on successful login. The token is included in the Authorization header of subsequent requests and validated by a security filter before a request reaches any controller.

3.3.2 Role-Based Access Control

Each authenticated request carries the user's role (Employee, Manager or HR/Admin), and Spring Security method- and endpoint-level configuration restricts access to role-appropriate functionality — for example, competency-definition endpoints are restricted to HR/Admin users, while profile and skill-inventory endpoints are scoped to the authenticated employee unless the requester has a manager or administrator role.

3.4 Backend Application Layer

The backend is organized into service classes aligned with the platform's twelve functional modules — authentication, profile and skill inventory, competency benchmarking, knowledge gap analysis, training recommendation, mentorship, learning progress, assessment, notification and analytics/reporting. Each service follows the same layered pattern: a REST controller receives the request, delegates to a service class containing business logic, and the service uses Spring Data JPA repositories to read and write PostgreSQL data through Hibernate-mapped entities.

3.5 Data Storage Layer

All persistent data is stored in a single PostgreSQL database, accessed exclusively through Spring Data JPA repositories. There is no separate caching layer or search index; list, filter and search operations (for example, searching employees by skill) are implemented as JPA queries against PostgreSQL directly. The database design is described in detail in Chapter 4.

3.6 Control Flow

Figure 3.2 illustrates the path of a typical request. A user interaction in a React component triggers an Axios call to the backend; the request passes through the Spring Security filter for JWT validation and role checking before reaching the relevant REST controller; the controller delegates to its service layer, which applies business logic and uses a repository to read or write PostgreSQL data; the result is mapped to a response DTO and returned as JSON, which the frontend uses to update component state and re-render the UI.

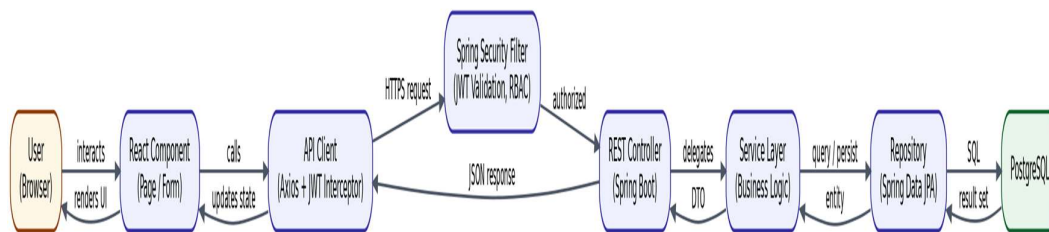


Figure 3.2: Control Flow and Component Interaction Diagram

3.7 Data Flow: Gap Analysis to Recommendation

Figure 3.3 shows how data moves through the platform's core analytical cycle. Employee skill-inventory entries and role competency requirements are the two inputs to the Knowledge Gap Analysis engine, which computes and stores knowledge-gap records. These gap records feed the rule-based Training Recommendation engine, which matches them against the training-program catalog to produce recommendations; enrollment and learning-progress records are

then generated as employees act on those recommendations. Gap records, enrollment status and progress updates all feed the in-application Notification service and the Analytics Dashboard, giving employees, managers and HR a consistent, connected view of the same underlying data.

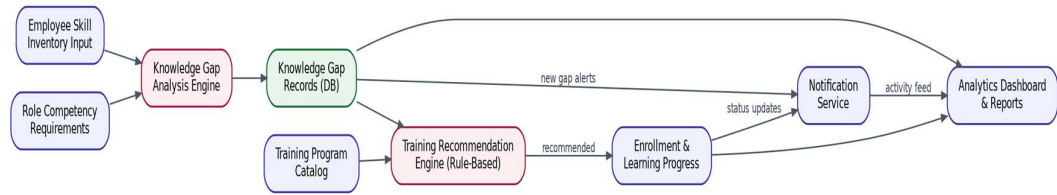


Figure 3.3: Data Flow Diagram for Knowledge Gap Analysis and Training Recommendation

3.8 Communication Model

All communication between the frontend and backend, and all processing within the backend, is synchronous request/response over REST. The platform does not use WebSocket connections, a message broker, or any background/asynchronous job processing; updates such as new notifications or recalculated gaps are produced as part of the request that triggers them and become visible the next time the relevant page is loaded or refreshed.

4. DATABASE AND DATA DESIGN

4.1 Database Overview

The platform uses a single PostgreSQL relational database, accessed through Spring Data JPA and Hibernate. The schema is designed around a central Employee Profile entity that links a user account to organizational context (department, role) and to the skill, gap, training, assessment, mentorship and notification data associated with that employee.

4.2 Entity Relationship Diagram

Figure 4.1 shows the core entities and their relationships. Departments contain roles; roles and departments are both referenced by employee profiles; profiles are linked to skill, gap, assessment, enrollment and mentorship records; and skills act as the common reference point connecting an employee's actual proficiency to the competency requirements defined for their role and to the training catalog used for recommendations.

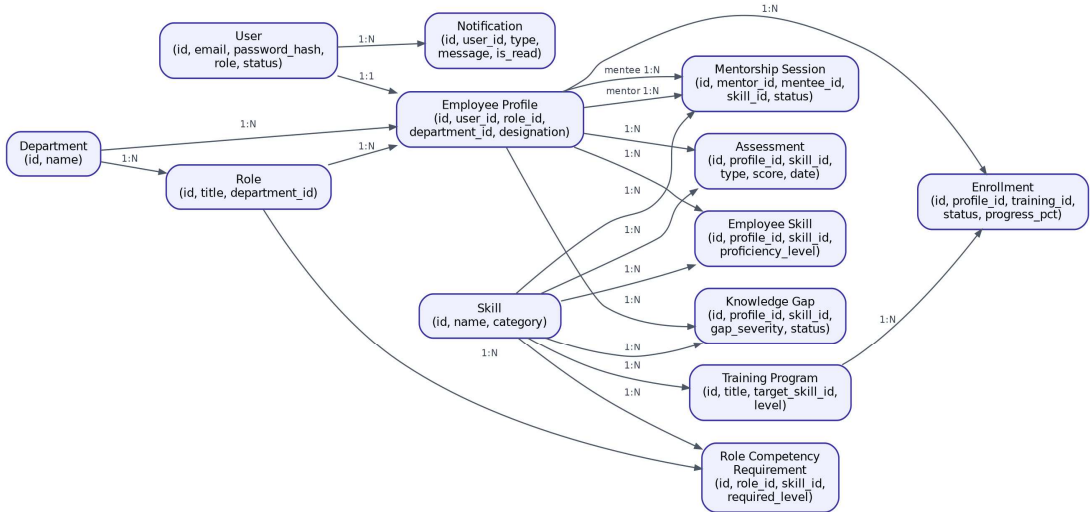


Figure 4.1: Entity Relationship Diagram of the Platform Database

4.3 Core Database Entities

Entity	Purpose
User	Stores login credentials, assigned role and account status.
Department	Represents an organizational department.
Role	Represents a job role within a department.
Employee Profile	Links a user to their department, role and profile details.
Skill	Master list of skills tracked by the platform.
Employee Skill	An employee's proficiency level for a given skill.
Role Competency Requirement	Required proficiency level for a skill within a given role.
Knowledge Gap	Computed gap between an employee's actual and required skill level, with severity and status.

Entity	Purpose
Training Program	A training/learning resource associated with a target skill.
Enrollment	An employee's enrollment and progress against a training program.
Assessment	Self, peer or manager assessment scores for a skill.
Mentorship Session	A mentor-mentee connection around a specific skill.
Notification	An in-application notification generated for a user.

Table 4.1: Core Database Entities

4.4 User and Profile Data

The User entity stores authentication data (email, hashed password, role) and is linked one-to-one with an Employee Profile that holds organizational details such as department, role/designation and contact information. Separating authentication data from profile data keeps the security-sensitive fields isolated from the broader profile data used throughout the rest of the application.

4.5 Skills and Competency Data

Skills are stored in a master Skill table shared across the platform. An employee's proficiency in a skill is recorded in the Employee Skill table, while the level expected for a given role is recorded separately in the Role Competency Requirement table. Keeping actual and required proficiency in separate tables, both referencing the same Skill entity, is what allows the gap analysis engine to compute a gap as a straightforward comparison between the two.

4.6 Knowledge Gap Data

Each identified gap is stored as a Knowledge Gap record referencing the employee profile and the skill involved, together with a computed severity and a status field used to track whether the gap is open, being addressed through an active training enrollment, or resolved. This table is the primary link between the analysis side of the platform and the recommendation and reporting modules.

4.7 Training and Progress Data

Training Program records describe available training content and the skill they target. Enrollment records link an employee profile to a training program and track status and progress percentage over time, providing the data used by the Learning Progress module and by manager-facing progress views.

4.8 Assessment, Mentorship and Notification Data

Assessment records capture self, peer or manager evaluation of an employee's proficiency in a given skill and are used to refresh Employee Skill data and, in turn, recalculate gaps. Mentorship Session records connect a mentor and mentee profile around a specific skill.

Notification records store the recipient user, type, message content and read status, and are queried directly by the frontend rather than pushed through any external channel.

4.9 Database Design Considerations

Foreign-key constraints are used throughout to enforce referential integrity between profiles, skills, gaps, enrollments and assessments, so that, for example, a gap record cannot exist without a valid profile and skill reference. The schema favors normalized tables with the Skill entity as a shared reference point, which avoids duplicating skill and proficiency definitions across the gap, competency and training-recommendation logic.

5. MODULE-WISE IMPLEMENTATION

The platform is organized into twelve functional modules, all of which were implemented and integrated into the working application. Each module below is described in terms of its purpose, key functionality, frontend/backend responsibility and outcome, without repeating the architecture and data-model details already covered in Chapters 3 and 4.

5.1 User Authentication and RBAC

Provides secure login and role-based access control so that each user only sees and can act on functionality appropriate to their role (Employee, Manager, HR/Admin).

Key Functionality

- Email/password login issuing a JWT access token.
- Role-aware routing and navigation on the frontend.
- Endpoint-level authorization enforced by Spring Security.
- Token-expiry handling via a 401 response interceptor in the API client.

Frontend / Backend Responsibility

Frontend: Login/registration pages, AuthContext for session state, protected routes based on role.

Backend: Spring Security configuration, JWT generation/validation filter, user and role entities.

Outcome

Users can only authenticate through a single, consistent flow and are restricted to role-appropriate pages and API endpoints throughout the platform.

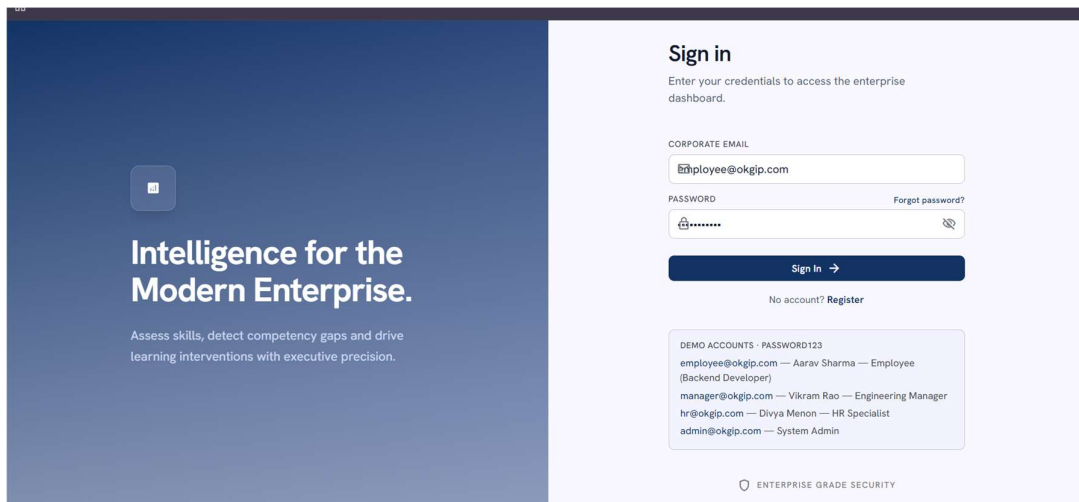


Figure 5.1: Application login screen with role-based demo accounts

5.2 Employee Profile and Skill Inventory

Allows employees to maintain their profile details and a structured inventory of their skills with proficiency levels, forming the base data used by every downstream module.

Key Functionality

- Profile creation and editing (department, role, designation).
- Adding, editing and removing skills with a proficiency level per skill.
- Department and role mapping used for competency comparison.

Frontend / Backend Responsibility

Frontend: Profile and skill-inventory forms, skill list/badge components.

Backend: Profile and Employee Skill REST endpoints and JPA entities linked to the Skill master table.

Outcome

Every employee has a queryable, structured skill inventory that the competency and gap-analysis modules can compare against role requirements.

The screenshot displays the 'My skill inventory' interface for a user named Aarav Sharma, a Backend Developer in the Engineering department. The page includes a sidebar with navigation options like Dashboard, Skill Profile, Competencies, Gap Analysis, Learning, Assessments, Knowledge, Analytics, and Notifications. The main content area is titled 'My skill inventory' and contains a 'Skill self-assessment' section. Below this, there is a table with columns for Skill, Category, Proficiency, and Source. The table lists several skills with their respective proficiency levels and sources.

Skill	Category	Proficiency	Source
Java	Backend	INTERMEDIATE	SELF
Spring Boot	Backend	BEGINNER	SELF
Spring Security	Backend	UNAWARE	SELF
SQL & PostgreSQL	Data	INTERMEDIATE	PEER
React.js	Frontend	UNAWARE	—
REST API Design	Architecture	INTERMEDIATE	SELF
Docker & CI/CD	DevOps	BEGINNER	SELF

Figure 5.2: Employee skill inventory with self-assessed proficiency levels

5.3 Competency Framework and Role Benchmarking

Lets HR/Administrators define the skills and proficiency levels expected for each role and department, forming the benchmark against which employee skills are compared.

Key Functionality

- Definition of role-specific required skills and levels.
- Department-level competency requirements.
- Administrative screens for maintaining the competency framework.

Frontend / Backend Responsibility

Frontend: Admin forms for defining and editing role competency requirements.

Backend: Role Competency Requirement entity and endpoints, linked to Role and Skill.

Outcome

A maintained, queryable set of role/department benchmarks that the gap analysis engine uses as its comparison baseline.

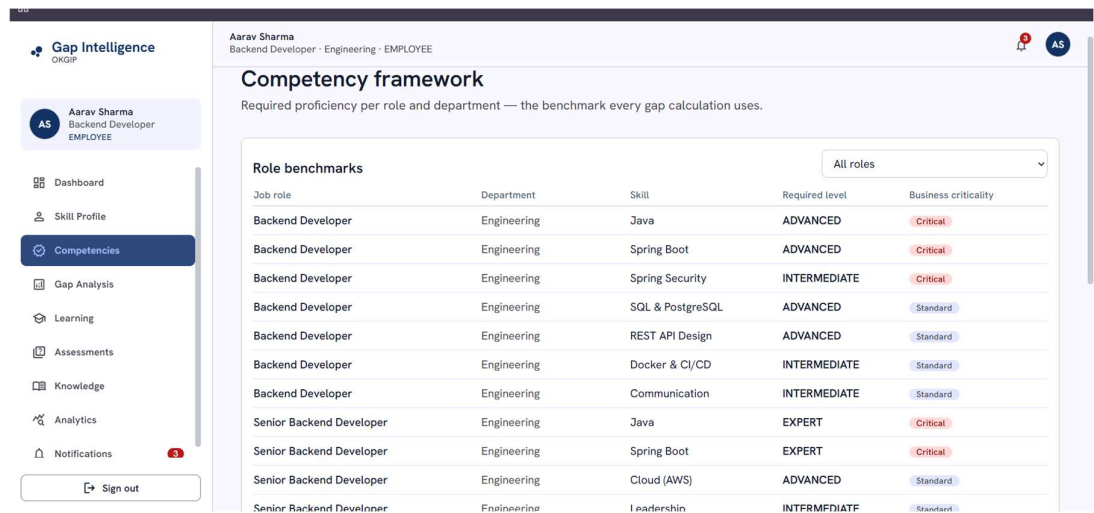


Figure 5.3: Competency framework showing required proficiency per role

5.4 Knowledge Gap Analysis

Compares each employee's actual skill levels against the competency requirements for their role and department to identify, classify and prioritize knowledge gaps.

Key Functionality

- Automatic computation of gaps by comparing Employee Skill against Role Competency Requirement records.
- Severity classification and gap status tracking (open / in progress / resolved).
- Individual, team and department-level gap views, including heatmap-style visualization.

Frontend / Backend Responsibility

Frontend: Gap-analysis views and heatmap visualization components for employees, managers and HR.

Backend: Gap-analysis service that computes and persists Knowledge Gap records and exposes aggregate views by team/department.

Outcome

Stakeholders can see, at each organizational level, exactly which skills are short of the required level and how severe each shortfall is.

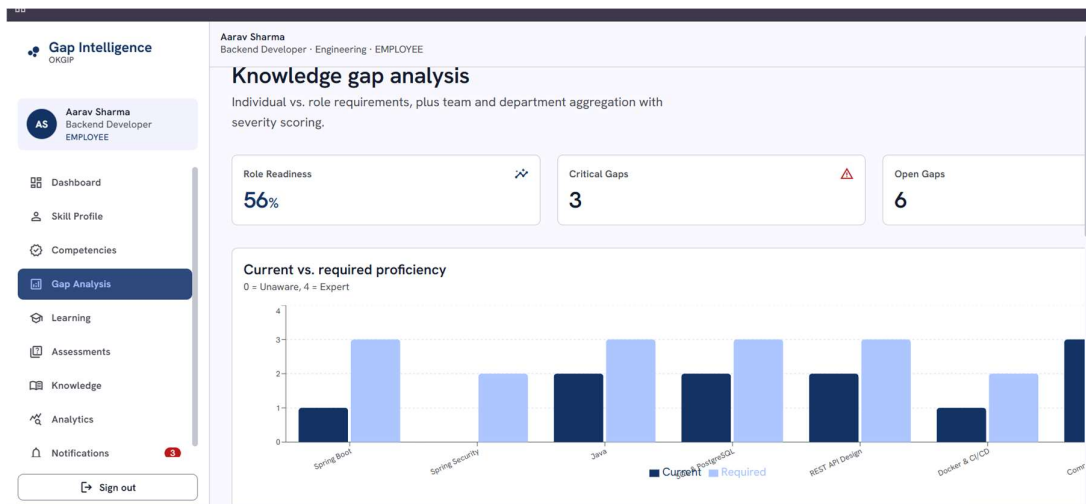


Figure 5.4: Current vs. required proficiency chart for an individual employee

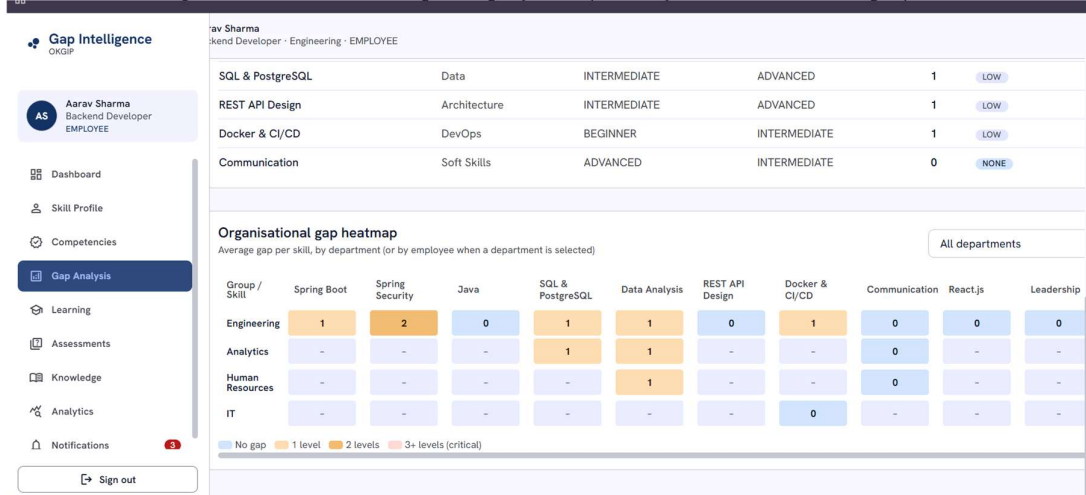


Figure 5.5: Organisational gap heatmap by department

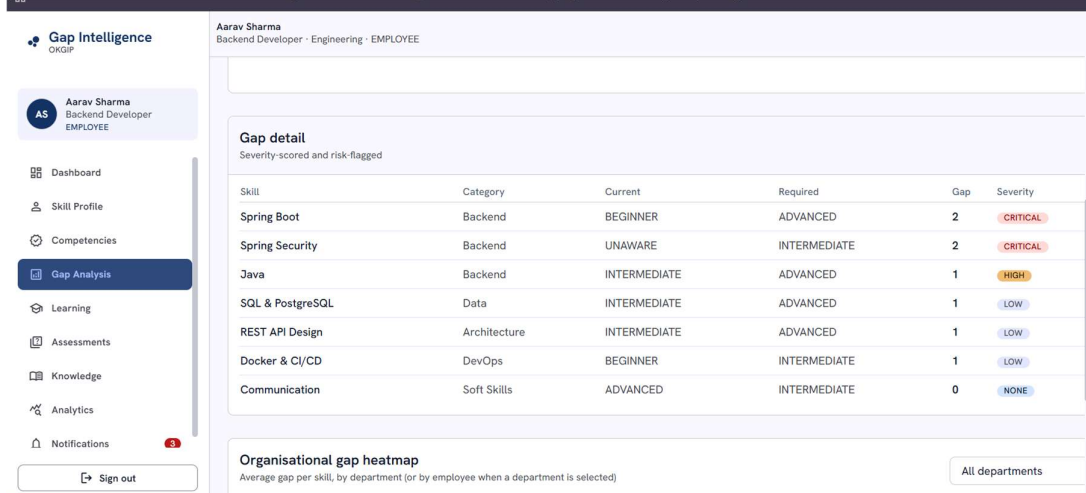


Figure 5.6: Gap detail table with automated severity scoring

5.5 Training Recommendation

Connects identified knowledge gaps to relevant training content using rule-based matching against the training-program catalog, rather than a generative AI/LLM-based approach.

Key Functionality

- Matching of open gaps to training programs targeting the same skill.
- Ranking of recommendations by gap severity and skill relevance.
- Learning-path style grouping of related recommended programs.

Frontend / Backend Responsibility

Frontend: Recommendation list/cards shown on the employee dashboard.

Backend: Recommendation service implementing the matching rules against Knowledge Gap and Training Program data.

Outcome

Employees receive a prioritized, gap-driven list of relevant training rather than having to search the catalog manually.

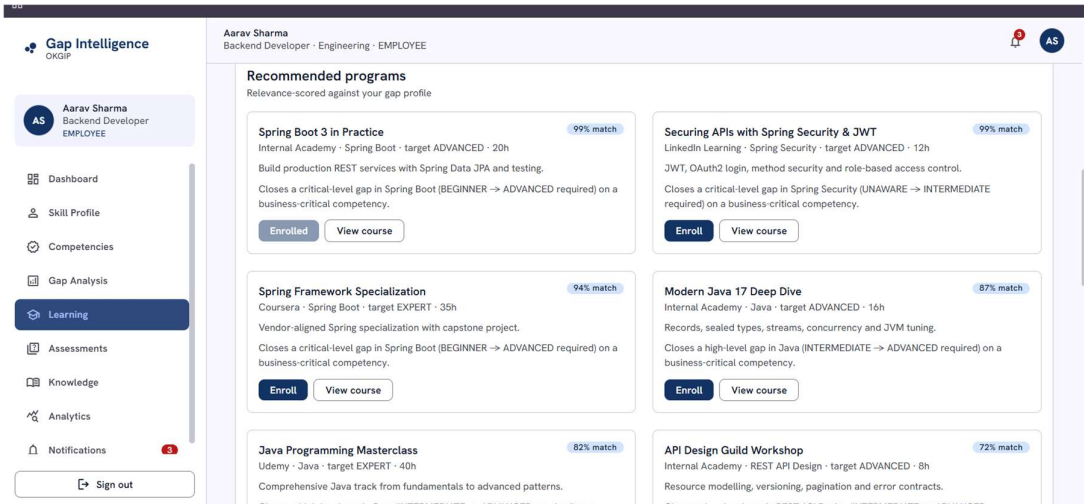


Figure 5.7: Recommended training programs ranked by gap-closure match

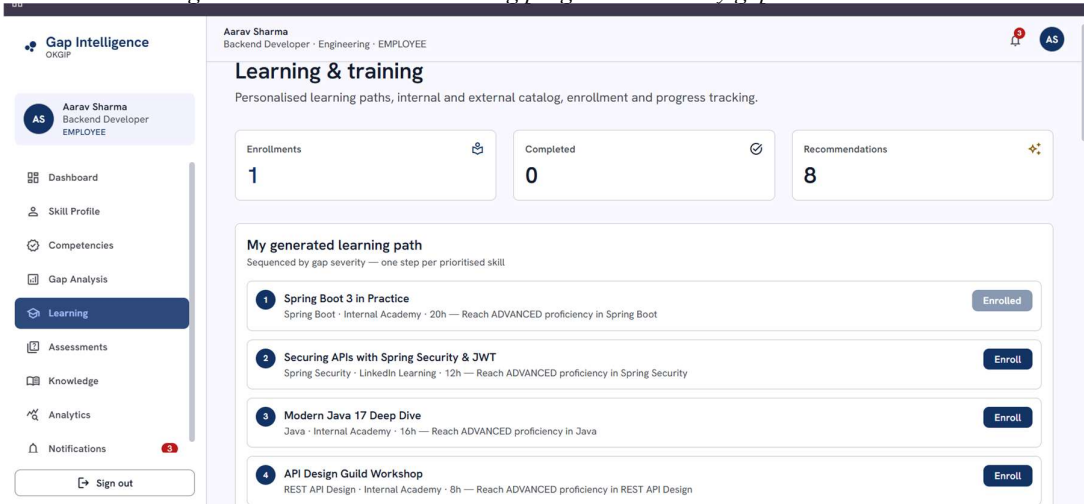


Figure 5.8: System-generated learning path sequenced by gap severity

5.6 Knowledge Sharing and Mentorship

Supports connecting employees who have a skill with employees who need it, for informal mentorship and knowledge transfer.

Key Functionality

- Matching of potential mentors and mentees around a specific skill.
- Mentorship session records tracking status.
- Directory-style view of employees with strong proficiency in a given skill.

Frontend / Backend Responsibility

Frontend: Mentor discovery and session-request screens.

Backend: Mentorship Session entity and endpoints linking mentor and mentee profiles to a skill.

Outcome

Employees can find and connect with colleagues who can help close a specific gap through direct knowledge transfer.

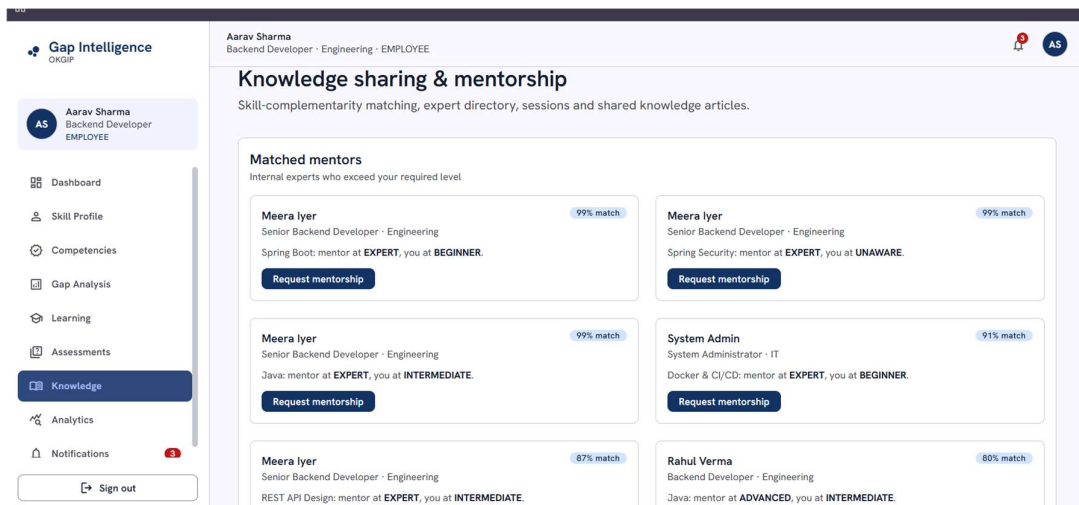


Figure 5.9: Matched mentors ranked by skill-complementarity score

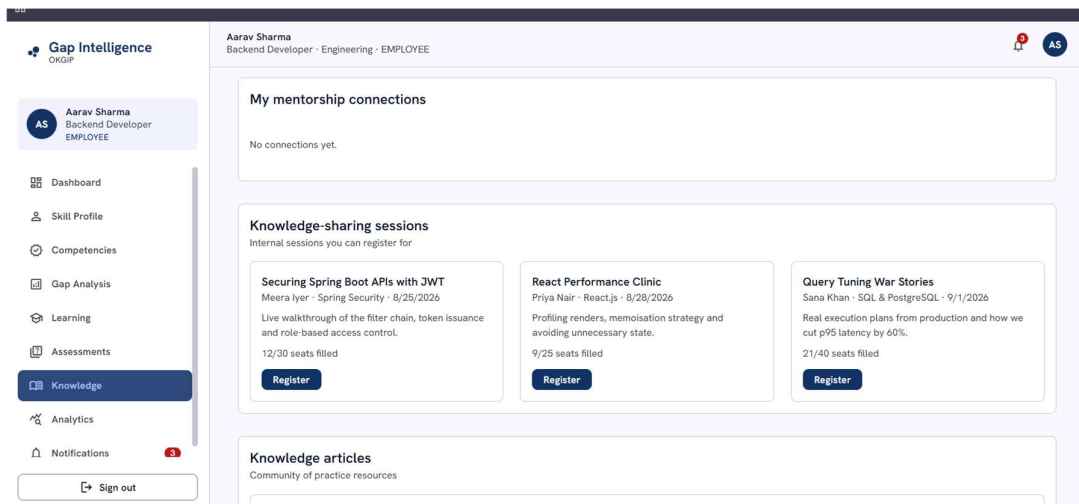


Figure 5.10: Knowledge-sharing sessions available for registration

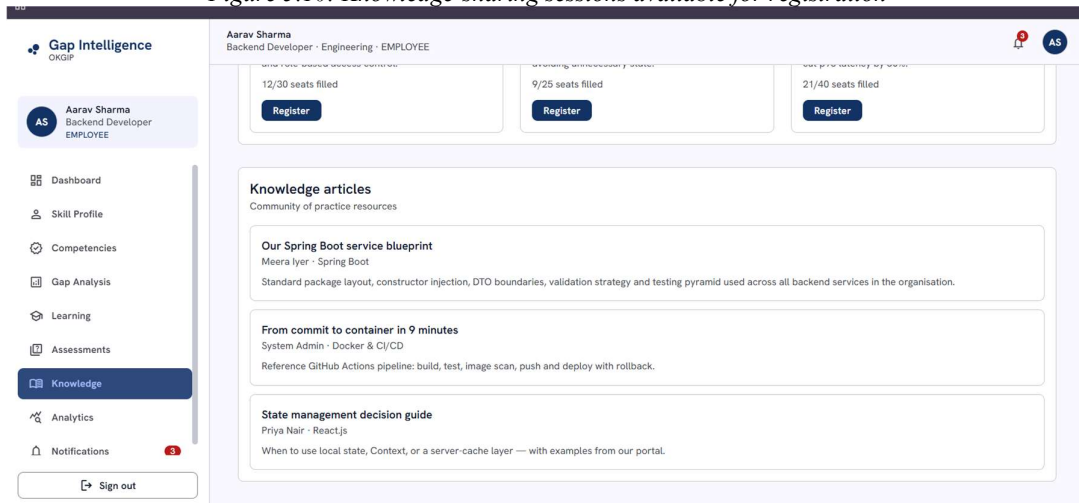


Figure 5.11: Knowledge articles shared across the community of practice

5.7 Learning Progress Tracking

Tracks an employee's enrollment and progress against recommended or self-selected training programs over time.

Key Functionality

- Enrollment in training programs with status (not started / in progress / completed).
- Progress percentage tracking per enrollment.
- Manager-facing visibility into team progress.

Frontend / Backend Responsibility

Frontend: Progress views for employees and managers, enrollment action controls.

Backend: Enrollment entity and service exposing progress-update endpoints.

Outcome

Both the employee and their manager can see whether recommended training is actually being completed, not just recommended.

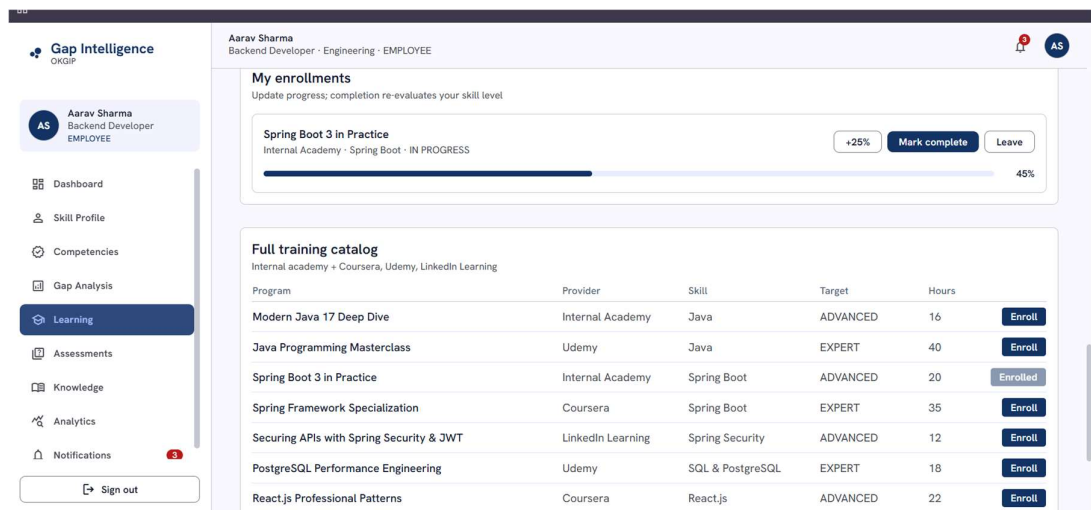


Figure 5.12: Enrollment progress tracking with completion controls

5.8 Assessment and Survey

Captures self, peer and manager assessment of an employee's skill proficiency, feeding updated proficiency data back into the skill inventory and gap analysis.

Key Functionality

- Self-assessment forms for employees.
- Manager evaluation of team-member proficiency.
- Assessment results used to refresh Employee Skill records, which in turn triggers gap recalculation.

Frontend / Backend Responsibility

Frontend: Assessment forms and results views.

Backend: Assessment entity and endpoints, integration point back into the Employee Skill and Gap Analysis services.

Outcome

Skill and gap data stays current by being periodically refreshed through structured assessment rather than only through manual profile edits.

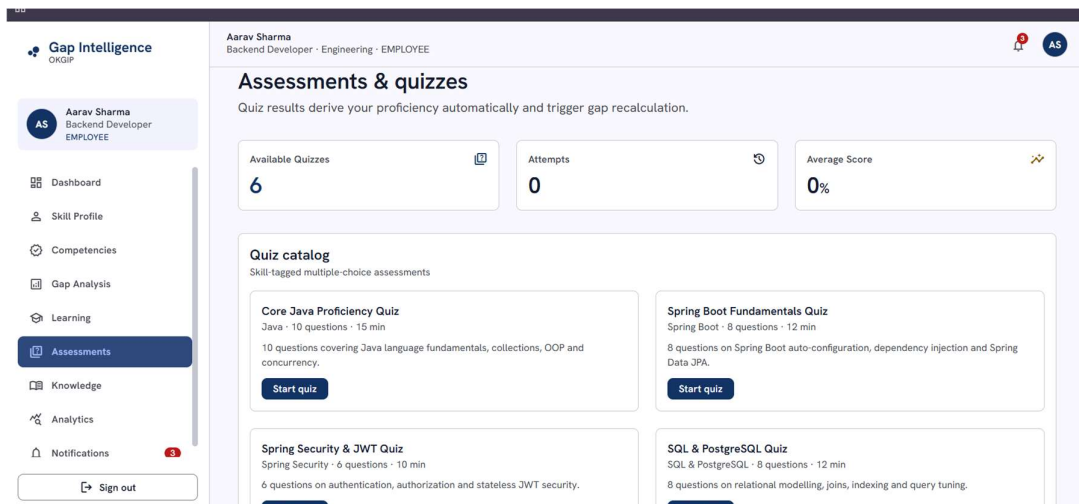


Figure 5.13: Skill-tagged assessment quiz catalog

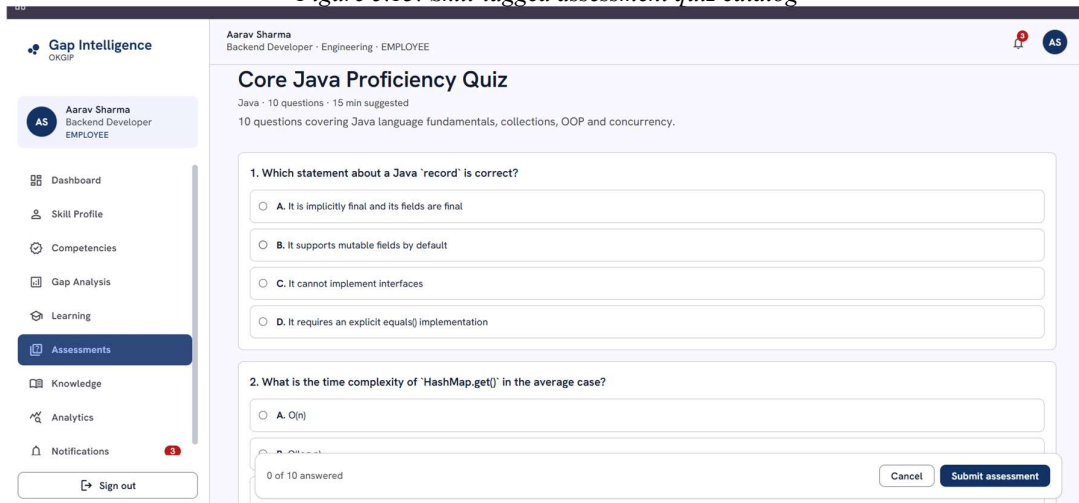


Figure 5.14: Core Java Proficiency quiz in progress

5.9 Notification

Keeps employees and managers informed of relevant platform events through notifications generated and stored within the application.

Key Functionality

- Notification records generated for events such as a new gap, a recommendation, or a mentorship request.
- In-app notification list with read/unread status.

Frontend / Backend Responsibility

Frontend: Notification bell/list component polling or fetching notification data.

Backend: Notification entity and endpoints; notifications are created directly by the relevant service (for example, the gap-analysis or mentorship service) rather than through an external messaging channel.

Outcome

Users have a single, consistent place within the platform to see relevant updates without relying on email or SMS integrations.

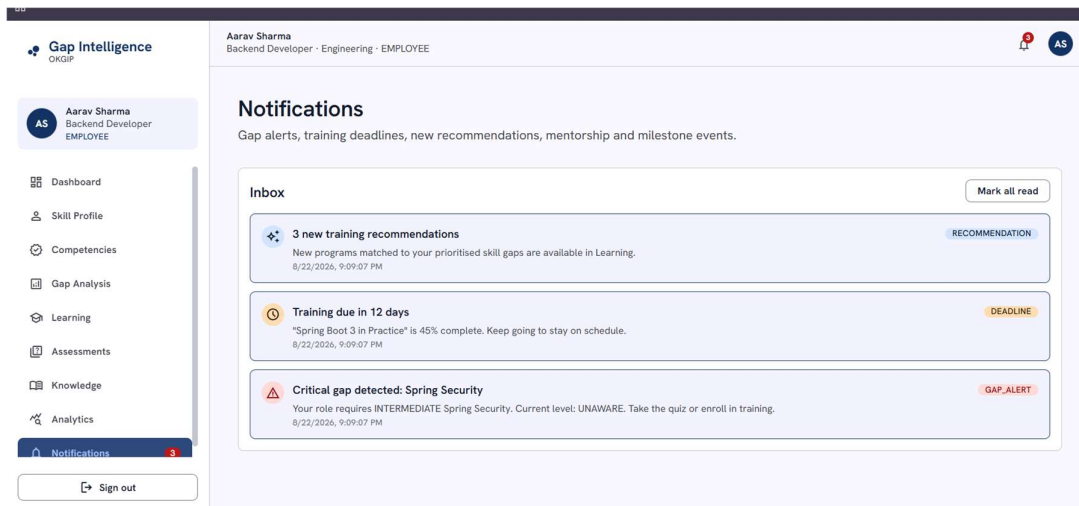


Figure 5.15: Notifications inbox showing gap alerts and training deadlines

5.10 Analytics Dashboard

Provides role-based dashboards summarizing skill, gap, training and progress data for employees, managers and HR/Admin users.

Key Functionality

- Employee dashboard summarizing personal skills, gaps and recommended training.
- Manager/department dashboard aggregating team-level gap and progress data.
- HR/Admin dashboard providing an organization-wide view.

Frontend / Backend Responsibility

Frontend: Dashboard pages using chart and stat-card components per role.

Backend: Analytics/reporting service aggregating data from the gap, enrollment and assessment tables.

Outcome

Each role gets a dashboard scoped to the data relevant to them, built on top of the same underlying gap and progress data.

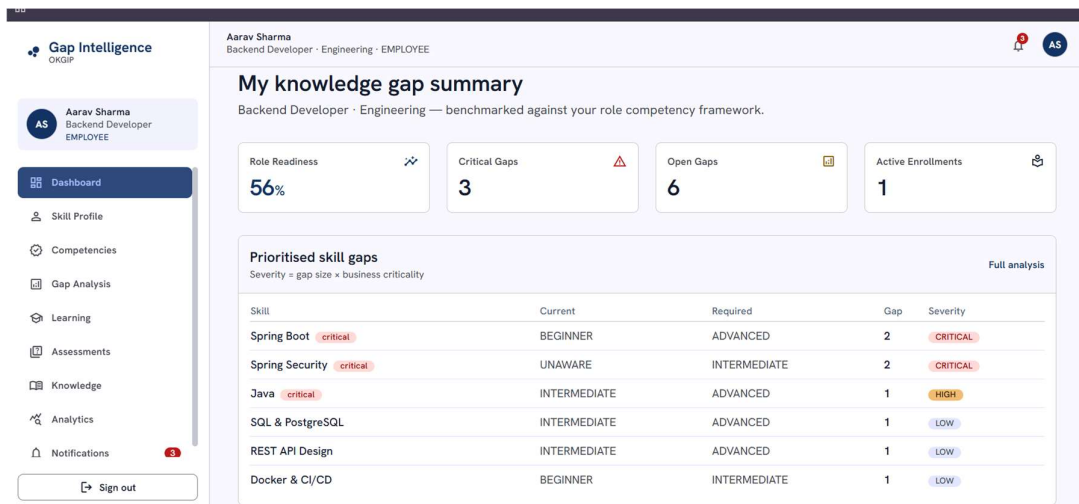


Figure 5.16: Employee dashboard: knowledge gap summary with prioritised gaps

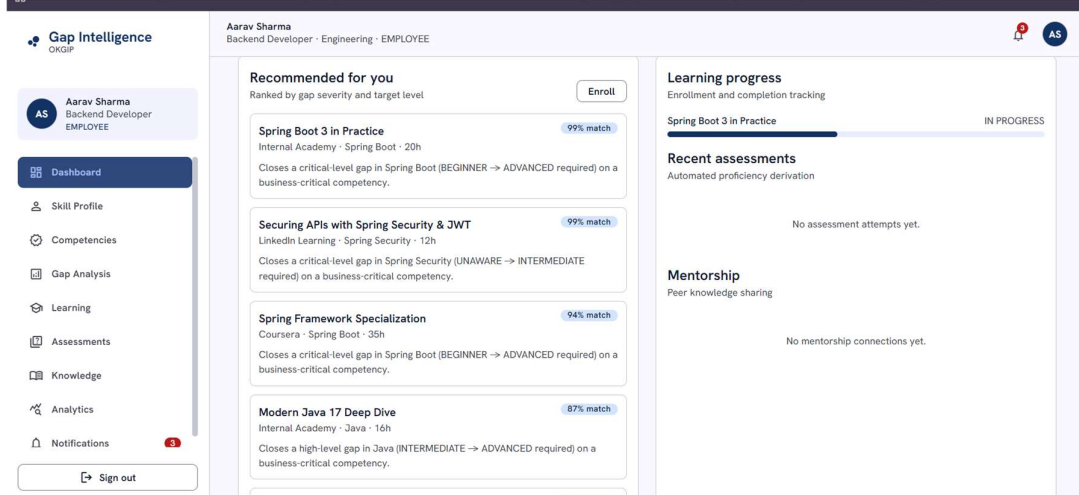


Figure 5.17: Employee dashboard: recommended training and learning progress

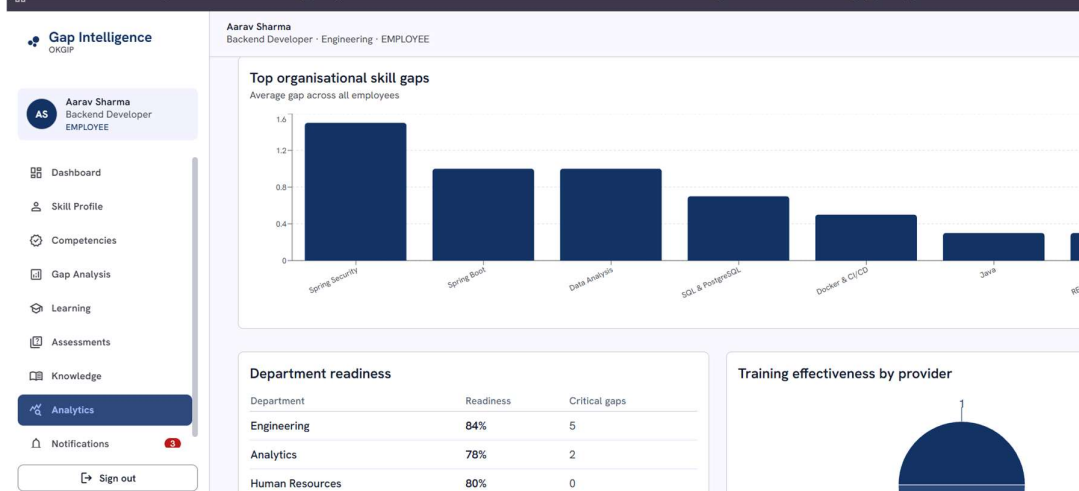


Figure 5.18: Analytics view: top organisational skill gaps and department readiness

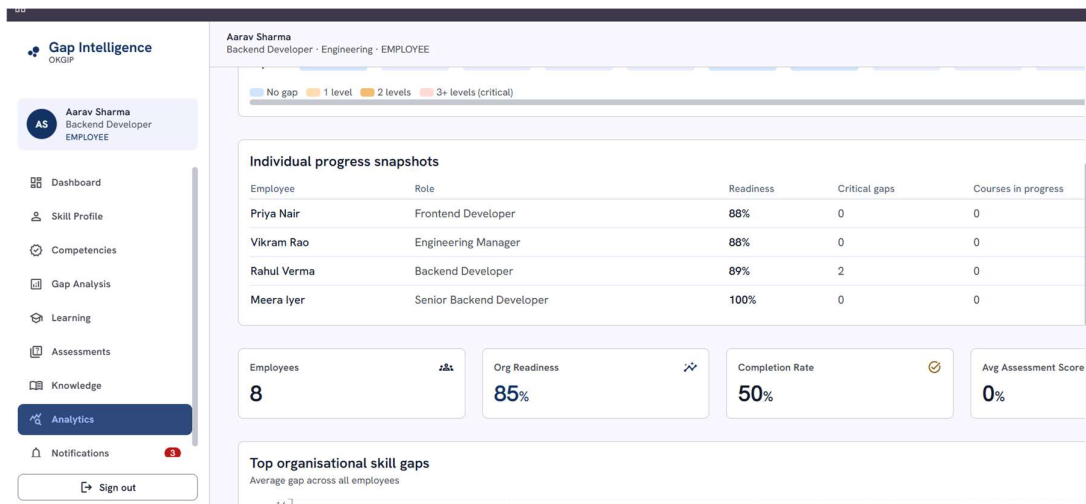


Figure 5.19: Analytics view: individual progress snapshots across the organisation

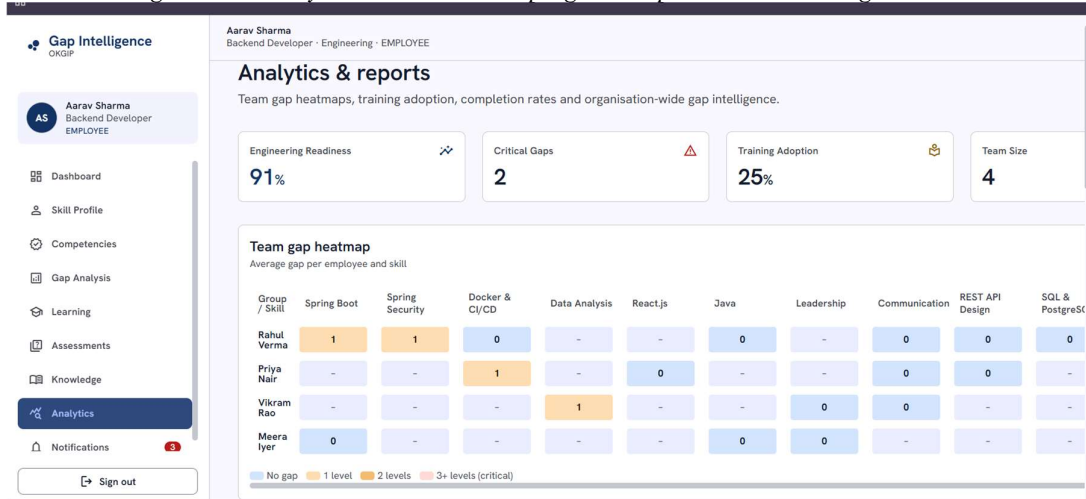


Figure 5.20: Analytics view: team gap heatmap by employee and skill

5.11 Reports and Export

Allows managers, HR and administrators to generate and export reports summarizing skill gaps and training outcomes at the individual, team and department level.

Key Functionality

- Individual and department-level gap reports.
- Training completion/outcome summaries.
- Export of report data for offline use.

Frontend / Backend Responsibility

Frontend: Report generation screens with export controls.

Backend: Reporting endpoints building on the same aggregation logic used by the analytics dashboard.

Outcome

Gap and training data can be extracted from the platform for offline review or sharing outside the application.

5.12 Final Integration, Testing and Deployment

Brings the eleven functional modules together into a single, consistent application, and covers the integration testing and local deployment carried out before final demonstration.

Key Functionality

- End-to-end integration of frontend and backend across all modules.
- Verification that data flows correctly from skill entry through gap analysis, recommendation, enrollment and reporting.
- Local build and run of the frontend (Vite dev/build) and backend (Spring Boot) for demonstration.

Frontend / Backend Responsibility

Frontend: Final integrated build of the React application.

Backend: Final integrated Spring Boot application connected to PostgreSQL.

Outcome

A working, integrated platform that was run and demonstrated locally, covering the complete skill-to-training lifecycle described in Chapter 3.

5.13 Module Summary

#	Module	Primary Role Users
1	User Authentication and RBAC	All roles
2	Employee Profile and Skill Inventory	Employee
3	Competency Framework and Role Benchmarking	HR/Admin
4	Knowledge Gap Analysis	Employee, Manager, HR/Admin
5	Training Recommendation	Employee
6	Knowledge Sharing and Mentorship	Employee
7	Learning Progress Tracking	Employee, Manager
8	Assessment and Survey	Employee, Manager
9	Notification	All roles
10	Analytics Dashboard	Employee, Manager, HR/Admin
11	Reports and Export	Manager, HR/Admin
12	Final Integration, Testing and Deployment	—

Table 5.1: Module Summary

6. TESTING, VALIDATION AND CODE QUALITY

6.1 Testing Strategy

Testing was carried out primarily through manual and functional verification of each module as it was built, together with API-level validation using Postman. Rather than reporting fabricated coverage percentages or test-case counts, this chapter describes the approach taken and the areas covered, in line with how the project was actually tested.

6.2 Functional Testing

Each module was manually exercised through its intended workflow after implementation — for example, creating a skill inventory entry and confirming it appeared correctly in gap analysis, or enrolling in a recommended training program and confirming the enrollment and progress records updated as expected. This was repeated for each of the twelve modules described in Chapter 5, and defects found during this process were fixed before the module was considered complete.

6.3 API Testing with Postman

Backend REST endpoints were validated directly using Postman during development, including authentication endpoints, role-restricted endpoints, and the core gap-analysis and recommendation endpoints, to confirm that requests returned the expected status codes and payloads and that unauthorized requests were correctly rejected.

6.4 Integration Testing

Once individual modules were functionally verified, integration testing focused on the connections between them — confirming that a change to an employee's skill inventory correctly propagated to gap analysis, that a newly identified gap correctly appeared in the recommendation engine's matching, and that enrollment and assessment updates were reflected in the analytics dashboard.

6.5 Security Validation

Authentication and role-based access control were checked by confirming that protected endpoints rejected requests without a valid JWT, and that endpoints restricted to a particular role (for example, HR/Admin-only competency-definition endpoints) rejected requests from users with other roles.

6.6 End-to-End Validation

Before the final demonstration, the full skill-to-training lifecycle — from skill entry, through gap computation, recommendation, enrollment, progress tracking and reporting — was walked through end-to-end using sample organizational data to confirm the modules operated correctly together.

6.7 Code Quality

The backend follows a layered controller-service-repository structure per module, and the frontend uses a shared component library, which kept module-specific logic isolated and made it easier to test and debug each module independently during development.

Layer	Testing Approach
Backend APIs	Manual endpoint validation using Postman.
Frontend Workflows	Manual functional walkthrough of each module's UI.
Cross-Module Integration	Manual verification that data changes propagate correctly between modules.
Security / RBAC	Manual verification of authentication and role-restricted access.
End-to-End	Full workflow walkthrough using sample data before demonstration.

Table 6.1: Testing Approach by Layer

7. CHALLENGES AND FUTURE ENHANCEMENTS

7.1 Development Challenges

7.1.1 Integrating Multiple Modules

With twelve interconnected modules sharing the same core skill and profile data, keeping the data model consistent as new modules were added required careful coordination between the frontend and backend teams, particularly around the Skill and Employee Profile entities that nearly every module depends on.

7.1.2 Frontend and Backend Integration

Aligning the React frontend's expected API response shapes with the Spring Boot backend's DTOs required iteration, especially for the gap-analysis and dashboard endpoints that aggregate data from several underlying tables.

7.1.3 Role-Based Access Design

Designing a role-access model that correctly scoped data visibility — for example, ensuring a manager could see their team's gaps but not another team's — required more careful endpoint-level authorization logic than a simple role check.

7.1.4 Gap Recalculation Consistency

Because assessments, profile edits and competency changes can all affect a gap's status, keeping Knowledge Gap records consistent whenever any of these upstream records changed required the gap-analysis service to be triggered from multiple places rather than only on a schedule.

7.2 Approaches Used to Address These Challenges

- A layered backend architecture (controller/service/repository) that isolated module-specific logic and reduced the impact of changes in one module on others.
- A shared, reusable frontend component library that kept UI patterns and data-fetching consistent across the twelve modules.
- Centralizing gap recalculation logic in a single service invoked from every place that can affect a gap, rather than duplicating the comparison logic.
- Iterative integration testing between frontend and backend after each module rather than only at the end of the project.

7.3 Future Enhancements

7.3.1 AI-Assisted Recommendations

The current training-recommendation logic is rule-based, matching gaps to the training catalog by target skill. A future enhancement identified by the team is to explore an LLM-based recommendation approach for more nuanced, context-aware suggestions.

7.3.2 Search and Performance at Scale

As the platform is intended for organization-wide use, introducing a dedicated search layer (such as Elasticsearch) and a caching layer (such as Redis) were identified as potential future additions once the platform is used with substantially larger datasets.

7.3.3 External Notification Channels

Extending notifications beyond the in-application list to email or other external channels was identified as a natural next step for keeping users informed outside active platform sessions.

7.3.4 Containerized and Cloud Deployment

The platform currently runs locally. Packaging the frontend and backend with Docker and deploying to a cloud environment was identified as the next step toward making the platform accessible outside a local development setup.

7.3.5 Additional Authentication Options

Adding Google OAuth2 as an alternative login method alongside the existing JWT-based authentication was identified as a usability improvement for future versions.

7.3.6 Mobile Application

A dedicated mobile application was identified as a future enhancement to extend platform access beyond the web interface.

7.4 Scalability Opportunities

Beyond the specific enhancements above, the team identified asynchronous processing (for example, via a message broker) for heavier analytics and notification workloads, and expanded reporting/analytics capabilities, as opportunities for scaling the platform to larger organizations in future iterations.

8. PROJECT OUTCOMES, TEAM CONTRIBUTIONS AND CONCLUSION

8.1 Project Outcomes

Team 1 delivered a working full-stack implementation of the Organizational Knowledge Gap Intelligence Platform covering all twelve planned modules described in Chapter 5, integrated end-to-end and demonstrated locally using representative organizational data. The platform successfully connects skill data capture, competency benchmarking, gap analysis, training recommendation, learning-progress tracking, mentorship, assessment, notification and reporting into a single, coherent application.

8.2 Team Structure

The project was carried out by a five-member team under Infosys Springboard Internship 7.0, organized around database/integration, frontend and backend responsibilities as shown in Table 8.1.

Team Member	Primary Responsibility
Sandhya Oni	Team Lead — Database and Integration
Likhith Sai Bandaru	Frontend Development
Roshan Kullu	Frontend Development
Harshitha Shri L	Backend Development
Sriranjani N	Backend Development

Table 8.1: Team Responsibilities

8.3 Team Contributions

8.3.1 Database and Integration

Sandhya Oni led the team and was responsible for the PostgreSQL database design described in Chapter 4 and for coordinating integration between the frontend and backend across all twelve modules.

8.3.2 Frontend Development

Likhith Sai Bandaru and Roshan Kullu were responsible for the React.js frontend, including the component library, role-specific pages and navigation, and the API integration layer connecting the frontend to backend services.

8.3.3 Backend Development

Harshitha Shri L and Sriranjani N were responsible for the Spring Boot backend, including the service and repository layers for the platform's modules, authentication and authorization logic, and the REST API surface consumed by the frontend.

8.4 Technical Learning Outcomes

- Practical experience designing and building a multi-module full-stack application with a shared relational data model.
- Hands-on experience with Spring Boot, Spring Security/JWT and Spring Data JPA on the backend.
- Hands-on experience with React.js, component-driven frontend architecture and REST API integration.
- Experience coordinating frontend/backend integration and manual/API-level testing across a team.

8.5 Organizational Value

The platform demonstrates how skill data, competency requirements and learning interventions can be brought together into a single system rather than managed as disconnected spreadsheets and manual reviews, giving managers and HR a more direct, data-backed view of where workforce competency gaps exist and whether training activity is addressing them.

8.6 Conclusion

The Organizational Knowledge Gap Intelligence Platform, developed by Team 1 as part of Infosys Springboard Internship 7.0, met its core objective of connecting employee skill data to competency benchmarks, automated gap analysis and training recommendations within a single full-stack application. All twelve planned modules were implemented and integrated, and the platform was validated through functional, API-level and end-to-end testing before final demonstration. The technologies not implemented in this version — including Elasticsearch, Redis, a message broker, Docker/cloud deployment, Google OAuth2 and external notification channels — have been documented as future enhancements rather than presented as part of the delivered system, in keeping with the actual scope of what the team built.

9. REFERENCES AND APPENDICES

9.1 References

The following official documentation sources correspond to the technologies actually used in the implemented platform, as described in Chapter 2.

- React documentation — react.dev
- React Router documentation — reactrouter.com
- Tailwind CSS documentation — tailwindcss.com
- Axios documentation — axios-http.com
- Spring Boot documentation — spring.io/projects/spring-boot
- Spring Security documentation — spring.io/projects/spring-security
- Spring Data JPA documentation — spring.io/projects/spring-data-jpa
- Hibernate ORM documentation — hibernate.org
- PostgreSQL documentation — postgresql.org/docs
- Postman documentation — learning.postman.com

9.2 Appendix A: System Architecture Diagram

Refer to Figure 3.1 in Chapter 3 for the full system architecture diagram.

9.3 Appendix B: Entity Relationship Diagram

Refer to Figure 4.1 in Chapter 4 for the full entity relationship diagram.

9.4 Appendix C: Control Flow and Data Flow Diagrams

Refer to Figures 3.2 and 3.3 in Chapter 3 for the control flow and data flow diagrams.

9.5 Appendix D: Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
JPA	Java Persistence API
JWT	JSON Web Token
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
HR	Human Resources

9.6 Appendix E: Team Information

Refer to Table 8.1 in Chapter 8 for full team member and responsibility information.